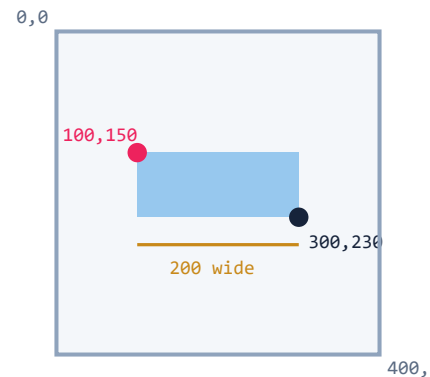


rect() — four numbers, two jobs

```
rect(x, y, width, height);
//  \___/  \_____/
//  top-left    how big
```

The last two numbers are a **size**, not another corner. To find the edges, **add**:

```
rect(100, 150, 200, 80)
right edge = 100 + 200 = 300
bottom edge = 150 + 80 = 230
```

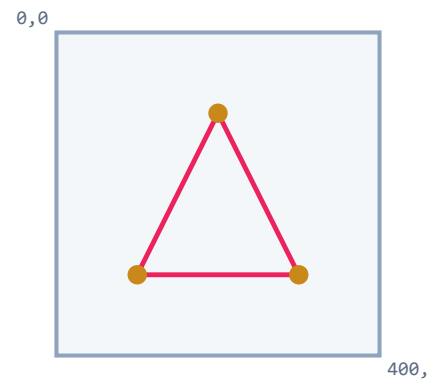


triangle() — three corners, one call

```
triangle(x1, y1, x2, y2, x3, y3);
```

Six numbers, read as **three corners**. It closes itself — there is no gap to get wrong.

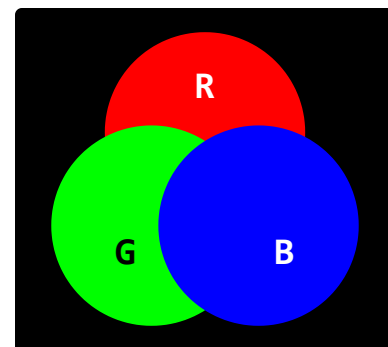
The same shape with `line()` takes three calls and nine numbers, because every corner has to be written twice.



Color

CALL	RESULT
<code>fill(200)</code>	☐ gray, one number
<code>fill(255, 0, 0)</code>	☐ red
<code>fill(0, 255, 0)</code>	☐ green
<code>fill(0, 0, 255)</code>	☐ blue
<code>fill(255, 255, 0)</code>	☐ yellow
<code>fill(0, 0, 0)</code>	☐ black
<code>fill(255, 255, 255)</code>	☐ white

One number is gray. Three numbers are red, green, blue — each 0 to 255.



light adds — red + green is yellow

`fill()` is a **setting**, not part of a shape. It applies to everything drawn after it, until you call `fill()` again. One `fill()` and four shapes gives you four shapes the same color. `stroke()` works the same way.

Order matters. Later shapes cover earlier ones. Reading your `draw()` top to bottom is the order things get painted — which is the same as distance from the viewer. If a shape vanished, it is usually *covered*, not missing. And anything drawn above `background()` is erased every frame.